

บทที่ 2

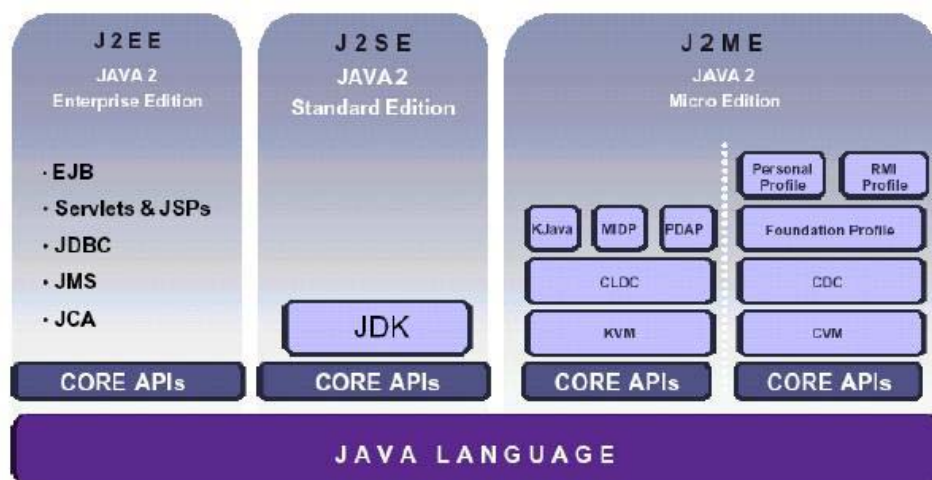
JAVA

2.1 แพลตฟอร์มจาวา

ในช่วงเวลาระยะหลังที่ผ่านมาจาวาได้กลายเป็นแพลตฟอร์มการพัฒนาแอปพลิเคชันแบบเชิงวัตถุ (object-oriented) ในอุปกรณ์และงานต่าง ๆ อย่างเต็มตัว นับตั้งแต่แอปพลิเคชันเซิร์ฟเวอร์ระดับองค์กร เครื่องคอมพิวเตอร์แบบตั้งโต๊ะทั่วไป จนถึงแอปพลิเคชันฝังตัวสำหรับอุปกรณ์ขนาดเล็ก

แพลตฟอร์มจาวา 2 ที่ใช้งานอยู่ในขณะนี้ มี 3 รุ่นด้วยกัน แต่ละรุ่นเน้นนำไปใช้กับเฉพาะกลุ่มแอปพลิเคชัน ดังนี้

- Java 2 Enterprise Edition (J2EE) ใช้งานกับแอปพลิเคชันบนเครื่องเซิร์ฟเวอร์สำหรับองค์กรที่รองรับระบบงานใหญ่ ๆ และไคลเอนต์จำนวนมาก
- Java 2 Standard Edition (J2SE) ใช้งานกับแอปพลิเคชันบนเครื่องคอมพิวเตอร์แบบตั้งโต๊ะทั่วไป
- Java 2 Micro Edition (J2ME) ใช้งานกับแอปพลิเคชันรุ่นใหม่ซึ่งเน้นกลุ่มผู้ใช้อุปกรณ์อิเล็กทรอนิกส์และอุปกรณ์ฝังตัว (eMbedded)



รูปที่ 2-1 แสดงแพลตฟอร์มจาวา 2 ทั้ง 3 รุ่น ที่ใช้งานอยู่ในขณะนี้

จาวาแต่ละรุ่นจะมีเวอร์ชวลแมชีน (Virtual machine) เฉพาะซึ่งปรับแต่งเป็นพิเศษ เพื่อให้สนับสนุนการทำงานของแอปพลิเคชันที่จะนำไปใช้ได้ดียิ่งขึ้น HotSpot VM เป็น Virtual machine ของจาวาที่ปรับแต่งเพื่อเพิ่มประสิทธิภาพของแอปพลิเคชันบนเครื่องเซิร์ฟเวอร์ที่เขียนด้วย J2EE

JVM เป็น Virtual machine สำหรับจาวาทั่วไปที่ปรับแต่งเพื่อใช้งานกับแอปพลิเคชันบนเครื่องคอมพิวเตอร์แบบตั้งโต๊ะ นอกจากนี้ HotSpot VM ยังสามารถทำงานเข้ากับแอปพลิเคชันที่เขียนด้วย JSE ได้เช่นกัน

Virtual Machine ที่ออกแบบมาเพื่อใช้งานกับ J2ME มี 2 ประเภทด้วยกัน คือ C Virtual Machine <CVM> และ K Virtual Machine <KVM> ซึ่งมีขนาดกะทัดรัดและใช้ทรัพยากรของระบบน้อยกว่า HotSpot VM และ JVM

2.2 จาวาทูโมโครเอดิชัน (J2ME™ Technology)

J2ME เป็นแพลตฟอร์มจาวาที่ออกแบบมาเพื่อใช้กับแอปพลิเคชันที่ทำงานบนอุปกรณ์ขนาดเล็ก เช่น โทรศัพท์เคลื่อนที่ พีดีเอ โทรศัพท์พร้อมจอภาพที่เชื่อมต่อกับอินเทอร์เน็ต โทรศัพท์ดิจิทัลขนาดเล็ก อุปกรณ์บันทึกและระบบนำทางในรถยนต์ สวิตช์ในระบบเครือข่าย คอมโพเนนต์ของเครื่องใช้อำนวยความสะดวกภายในบ้าน ฯลฯ

J2ME ได้นำโครงสร้างแบบโมดูลที่มีความยืดหยุ่นสูงเข้ามาใช้ เพื่อให้สามารถสนับสนุนการทำงานอุปกรณ์หลากหลายประเภท รูปที่ 2.1 แสดงให้เห็นว่า J2ME กำหนดชั้นของซอฟต์แวร์ไว้ 3 เลเยอร์ด้วยกัน โดยเลเยอร์ทั้งหมดจะอยู่เหนือชั้นระบบปฏิบัติการของอุปกรณ์ ดังนี้

- เลเยอร์ Java Virtual Machine เป็นเลเยอร์ของ Java Virtual Machine ปรับแต่งให้เข้ากับระบบปฏิบัติการของอุปกรณ์ และรองรับแต่ละ configuration ของ J2ME จากรูปที่ 2.1 Virtual Machine ของ J2ME ได้แก่ CVM และ KVM
- เลเยอร์ Configuration เป็นเลเยอร์ของคอนฟิกูเรชันของ J2ME ซึ่งกำหนดคลาสไลบรารี อุปกรณ์ใช้งานทั่วไปหรือกลุ่มอุปกรณ์ที่มีความต้องการหน่วยความจำและหน่วยประมวลผลใกล้เคียงกัน จากรูปที่ 2.1 Configuration ใน J2ME มี 2 ประเภทด้วยกัน คือ CDC < Connected Device Configuration > และ CLDC < Connected Limited Device Configuration >
- เลเยอร์ Profile เป็นเลเยอร์ที่สร้างเหนือเลเยอร์คอนฟิกูเรชัน โดยกำหนดคลาสไลบรารีเพื่อสนองตอบความต้องการของตลาดเฉพาะกลุ่ม จากรูปที่ 2.1 ตัวอย่างของ Profile ในเลเยอร์นี้ ได้แก่ PDAP, MIDP, Foundation Profile และ Personal Profile

2.2.1 คอนฟิกูเรชันใน J2ME

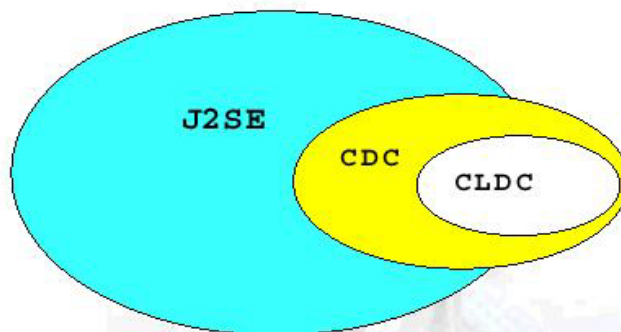
คอนฟิกูเรชันและโพรไฟล์เป็นองค์ประกอบหลักของ J2ME โดยมีจุดสำคัญ คือ เพื่อปรับแต่ง virtual machine และคลาสไลบรารีให้เหมาะสมกับอุปกรณ์แต่ละประเภท คอนฟิกูเรชัน คือชุดที่มีคุณสมบัติขั้นต่ำของ Java Virtual Machine และจาวาคลาสสำหรับอุปกรณ์แต่ละประเภท เป็นตัวแทนของอุปกรณ์ที่ลักษณะคล้ายคลึงกัน อาจกล่าวได้ว่า คอนฟิกูเรชันเป็นตัวกำหนดคุณสมบัติหรือไลบรารีกำหนดใน คอนฟิกูเรชันจะมีในทุกอุปกรณ์ที่อยู่ในกลุ่มเดียวกัน

ปัจจุบัน คอนฟิกูเรชันใน J2ME แบ่งเป็น 2 ประเภทคือ CDC (Connected Device Configuration) และ CLDC (Connected Limited Device Configuration) โดยมีเป้าหมายที่กลุ่มอุปกรณ์ 2 ประเภทซึ่งมีความต้องการหน่วยความจำและหน่วยประมวลผลพื้นฐานใกล้เคียงกัน

- CDC เป็นอุปกรณ์ใช้ร่วมกับอุปกรณ์อื่น ติดตั้งตายตัว และใช้เชื่อมต่อข้อมูล โดยปกติมักมีส่วนติดต่อกับผู้ใช้หลากหลายแบบ มีหน่วยความจำประมาณ 2-16 เมกกะไบต์ ใช้หน่วยประมวลผลแบบ 32 บิต หรือมากกว่าเชื่อมต่อเครือข่ายที่มีแบนด์วิดธ์สูงอย่าง

ต่อเนื่องโดยอาศัยพอร์ต TCP/IP ตัวอย่างอุปกรณ์ประเภทนี้ได้แก่ โทรศัพท์มือถือขนาดเล็ก อินเทอร์เน็ตทีวี โทรศัพท์พร้อมจอภาพที่เชื่อมต่อกับอินเทอร์เน็ต อุปกรณ์สื่อสารที่มีความซับซ้อนสูง อุปกรณ์บันทึกและระบบนำทางในรถยนต์

- CLDC เป็นอุปกรณ์ส่วนบุคคล พกพาได้ และใช้เชื่อมต่อข้อมูล โดยปรกติมักมีส่วนติดต่อกับผู้ใช้แบบง่ายๆ (เมื่อเทียบกับระบบบนเครื่องคอมพิวเตอร์แบบตั้งโต๊ะ) มีหน่วยความจำประมาณ 128 กิโลไบต์ – 1 เมกกะไบต์ ใช้หน่วยประมวลผลแบบ 16 หรือ 32 บิต เชื่อมต่อกับเครือข่ายที่มีแบนด์วิดธ์ต่ำเป็นระยะเวลาด้านๆ โดยไม่อาศัยพอร์ต TCP/IP ตัวอย่างอุปกรณ์ประเภทนี้ได้แก่ โทรศัพท์เคลื่อนที่แบบไม่ซับซ้อนมากนัก เพจเจอร์รับส่งข้อความ เครื่องปาล์มโอเอสแบบพกพา



รูปที่ 2-2 แสดงความสัมพันธ์ระหว่าง J2SE และคลาสไลบรารีใน CDC และ CLDC

ในชั้นของ Configuration มีคลาส 2 ประเภทด้วยกัน คือ คลาสที่นำมาจาก J2SE และที่ออกแบบเฉพาะอุปกรณ์ขนาดเล็ก คลาสที่นำมาจาก J2SE จะมีคุณสมบัติอย่างเดียวกันกับคลาสใน J2SE หรือเป็นซับคลาสของ J2SE เช่น แพ็กเกจ java.lang java.io และ java.util จากรูปที่ 2-2 แสดงความสัมพันธ์ระหว่าง J2SE และคลาสไลบรารีของ CDC และ CLDC จากภาพจะเห็นว่าคลาสใน CLDC ส่วนใหญ่ทำงานร่วมกับคลาสใน CDC ได้ดีเช่นเดียวกับกรณีของคลาสใน CLDC

ขณะเดียวกันคลาสที่ไม่ได้นำมาจาก J2SE และออกแบบเพื่อใช้เฉพาะอุปกรณ์มักทำงานร่วมกับ J2SE ได้ไม่ค่อยดี ใน CLDC คลาสเหล่านี้จะอยู่ในกลุ่มกรอบการติดต่อสื่อสารทั่วไป (Generic Connection Framework) โดยระบุไว้ในแพ็คเกจ javax.microedition.io

2.2.2 J2ME สำหรับอุปกรณ์ไร้สาย

J2ME ได้ให้กำเนิดแอปพลิเคชันยุคใหม่บนอุปกรณ์ไร้สาย ช่วยให้เกมแบบหลายผู้เล่นที่ทำงานผ่านอินเทอร์เน็ตการทำธุรกรรมทางโทรศัพท์มือถือ แอปพลิเคชันสำหรับองค์กรทั้งไคลเอ็นท์และเซิร์ฟเวอร์ เกิดขึ้นได้บนโทรศัพท์เคลื่อนที่ และเพจเจอร์รับส่งข้อความ MIDP CLDC และ KVM ได้กลายมาเป็นรากฐานในการพัฒนาจาวาแอปพลิเคชันบนอุปกรณ์ไร้สายยุคใหม่

สามารถแบ่งโครงสร้างออกเป็นเลเยอร์ต่าง ๆ จากล่างขึ้นบน ดังนี้

- เลเยอร์ฮาร์ดแวร์ MID หมายถึง ตัวโทรศัพท์เคลื่อนที่ (เช่น โมโตโรลา รุ่น iDEN 3000) หรือเพจเจอร์รับส่งข้อความ (เช่น RIM รุ่น Blzckberry 950)
- เลเยอร์ซอฟต์แวร์ของระบบที่ติดตั้งมากับอุปกรณ์ หมายถึง ระบบปฏิบัติการและไลบรารีของระบบที่บริษัทผู้ผลิตให้มา
- เลเยอร์ KVM เป็นส่วนที่เตรียม runtime environment ไว้ให้แอปพลิเคชันบนอุปกรณ์ไร้สาย
- เลเยอร์ CLDC เป็นส่วนที่เตรียม API หลักของจาวาให้แอปพลิเคชันบนอุปกรณ์ไร้สาย
- เลเยอร์ MIDP เป็นส่วนที่เตรียมไลบรารีสำหรับส่วนติดต่อกราฟฟิคกับผู้ใช้ และหน่วยเก็บข้อมูล pesistent storage ระบบเครือข่าย และไทม์เมอร์

นอกจากคลาสไลบรารีสำหรับ MIDP อาจเตรียมคลาสไลบรารีเฉพาะอุปกรณ์ไว้ให้นักพัฒนา เพื่อดึงความสามารถของฟังก์ชันที่มีอยู่แล้วไปใช้งานให้เกิดประโยชน์สูงสุด อาทิ การโทรศัพท์ การแชร์ข้อมูลกับแอปพลิเคชันที่ติดตั้งมาในเครื่อง (เช่น ปฏิทิน สมุดจดที่อยู่) การตรวจสอบข้อมูลอุปกรณ์ที่ผู้ผลิตเตรียมไว้มาใช้งาน จะช่วยเพิ่มความสามารถแก่แอปพลิเคชันบนอุปกรณ์ไร้สาย แต่ไม่สามารถเคลื่อนย้ายไปสู่อุปกรณ์อื่นที่ใช้ MIDP ได้ เนื่องจากคลาสที่นำมาใช้อยู่นอกเหนือขอบเขตของ MIDP

2.2.3 ความต้องการของระบบ

อุปกรณ์ไร้สายจะทำงานสนับสนุน J2ME ได้ดีเมื่อมีคุณสมบัติตรงตามข้อกำหนด หากต้องการให้ KVM ทำงานได้อย่างมีประสิทธิภาพ โดยใช้ไลบรารี CLDC จะต้องมีคุณสมบัติของระบบขั้นต่ำดังนี้

- มีหน่วยความจำ 160 - 512 กิโลไบต์ สำหรับสร้างแพลตฟอร์มจาวา
- มีหน่วยประมวลผลแบบ 16-32 บิตความเร็ว 25 เมกกะเฮิรตซ์
- ใช้พลังงานน้อย โดยมากมักทำงานโดยใช้แบตเตอรี่
- เชื่อมต่อกับเครือข่ายได้ในช่วงสั้นๆ (อาศัยระบบไร้สายเป็นส่วนใหญ่) แบนด์วิดท์ค่อนข้างจำกัด (ความเร็ว 9600 ไบต์ต่อวินาทีหรือน้อยกว่า)
- มีหน่วยความจำชั่วคราวขนาด 32 กิโลไบต์ สำหรับเก็บจาวา รันไทม์และหน่วยความจำของแอปเจ็ท

2.3 หลักการพัฒนาแอปพลิเคชันบน MIDP ด้วย J2ME

2.3.1 ไลบรารีของ CLDC

ไลบรารีของ CLDC เป็นไลบรารีของเครือข่ายและระบบชั้นสูงที่ไม่จำกัดเฉพาะอุปกรณ์ใดอุปกรณ์หนึ่ง ประกอบด้วยคลาส 2 ประเภท คือ คลาสที่เป็นซัพเซตของ J2SE และคลาสที่เกี่ยวกับการติดต่อสื่อสารทั่วไป (Generic Connection Framework) ของ CLDC

2.3.2 ชั้นคลาสของ J2SE

คลาสประเภทแรกของ CLDC เป็นชั้นคลาสของไลบรารี J2SE กำหนดไว้ในแพ็คเกจ java.lang java.util และ java.io โดยคลาสระบบและคลาสชนิดข้อมูลเข้ากันได้กับ J2SE และ J2EE และเพื่อให้เข้า

กันได้และเคลื่อนย้ายข้ามแพลตฟอร์มได้โดยสะดวก คลาสประเภทนี้จะใช้ชื่อแพ็คเกจตามชื่อคลาสอย่างเดียวกันใน J2SE หรือตามชื่อซัพคลาสของคลาสใน J2SE ที่เกี่ยวเนื่องกัน โดยไม่มีการเพิ่มเมธอด public หรือ protected หรือ final ซึ่งไม่มีในคลาสที่เกี่ยวข้องกันใน J2SE ความหมายของคลาสและเมธอดจึงไม่เปลี่ยนไป

2.3.3 คลาสที่มีเฉพาะใน CLDC

คลาสประเภทที่สองของ CLDC กำหนดไว้ในแพ็คเกจ javax.microedition.io คลาสเหล่านี้มีเฉพาะใน CLDC จึงไม่สามารถทำงานร่วมกับไลบรารีของ J2SE ได้ เป็นคลาสชั้นสูงเกี่ยวกับเรื่องทั่วไปของเครือข่าย กำหนดไว้ในแพ็คเกจ java.io และ java.net คลาสประเภทนี้จะเตรียมกรอบการติดต่อสื่อสารผ่านเครือข่ายให้กับอุปกรณ์ที่สนับสนุน J2ME โดยกรอบการสื่อสารนี้มักเรียกกันว่า Generic Connection Framework คลาสส่วนใหญ่ได้แก่อินเตอร์เฟซที่แทนการสื่อสารรูปแบบต่างๆ เช่น ซอกเกต (Socket) ดาต้าแกรม (Datagram) ซีเรียล (Serial) และ http ขึ้นอยู่กับว่าผู้ผลิตอุปกรณ์หรือผู้ให้บริการเครือข่ายจะติดตั้งอินเตอร์เฟซการเชื่อมต่อเหล่านี้เพียงบางตัวหรือครบทั้งหมด เมื่อพิจารณาจากความสามารถของอุปกรณ์และเครือข่าย

2.3.4 ประเภทของข้อมูลพื้นฐาน

CLDC สนับสนุนเฉพาะซัพเซตของชนิดข้อมูลที่นำมาจาก J2SE ซึ่งได้แก่ byte short int long char และ boolean แต่ไม่สนับสนุนข้อมูลประเภท float และ double เนื่องจากสาเหตุ 2 ประการ คือ อุปกรณ์เป้าหมายที่นำ CLDC ไปใช้ ส่วนใหญ่ไม่มีฮาร์ดแวร์รองรับตัวเลขทศนิยม และหากจะใช้ซอฟต์แวร์ช่วยก็จะต้องเสียค่าใช้จ่ายสูงมาก

นอกจากนี้ CLDC ยังกำหนดคลาส type wrapper สำหรับข้อมูลทุกประเภท ได้แก่

- Java.lang.Boolean
- Java.lang.Byte
- Java.lang.Character
- Java.lang.Integer
- Java.lang.Long
- Java.lang.Short

คลาส type wrapper กำหนดไว้ในจาวา เนื่องจากจาวามีระบบย่อยหลายระบบซึ่งทำงานได้เฉพาะกับออบเจกต์เท่านั้นในกรณีนี้ คุณสามารถสร้างออบเจกต์โดยใช้คลาส wrapper ซึ่งเก็บชนิดของข้อมูลที่นำมาจาก J2SE เอาไว้

2.3.5 ไลบรารีของ MIDP

ขณะที่ไลบรารีของ CLDC ช่วยสร้างฟังก์ชันที่ไม่จำกัดเฉพาะอุปกรณ์บางชนิด ไลบรารีของ MIDP กลับตรงข้ามได้แก่ การจัดการแอปพลิเคชันบนอุปกรณ์ ส่วนติดต่อกราฟฟิกกับผู้ใช้ทั้งแบบพื้นฐานและซับซ้อน พื้นที่เก็บข้อมูลแบบคงตัว (persistent storage) และความสามารถเพิ่มเติมบนเครือข่าย

2.3.6 คลาสโปรแกรมจัดการแอปพลิเคชัน

คลาสที่ติดต่อกับโปรแกรมจัดการแอปพลิเคชันบนอุปกรณ์ถูกกำหนดไว้ในแพ็คเกจ `java.microedition.midlet` แอปพลิเคชันทั้งหลายที่เขียนใน MIDP จะต้องขยายคลาส `MIDlet` ที่อยู่ในแพ็คเกจออกไป และจะต้องนำเมธอดทั้ง 3 ซึ่งได้แก่ `startApp()` `pauseApp()` และ `destroyApp()` เข้ามาใช้

2.3.7 คลาสของส่วนติดต่อกราฟิกกับผู้ใช้ (GUI Class)

ชุดเครื่องมือ Abstract Windowing Toolkit ใน J2SE ออกแบบมาเพื่อใช้งานกับแอปพลิเคชันบนเครื่องคอมพิวเตอร์แบบตั้งโต๊ะและไม่สามารถนำมาใช้กับอุปกรณ์ไร้สายได้ เนื่องจากต้องการทรัพยากรประเภทหน่วยความจำค่อนข้างมาก MIDP มีวิธีการที่ต่างกันในการกำหนดไลบรารีสำหรับแพ็คเกจส่วนติดต่อกราฟิกกับผู้ใช้ ด้วยการใช้ API ชั้นสูงซึ่งเน้นความสามารถในการเคลื่อนย้ายข้ามอุปกรณ์ และ API ชั้นพื้นฐานซึ่งเน้นองค์ประกอบกราฟิกเฉพาะอุปกรณ์และ input event ทั่วไป คลาส ที่เชื่อมต่อกับส่วนติดต่อกราฟิกกับผู้ใช้ และ event-handling กำหนดไว้ในแพ็คเกจ `java.microedition.lcdui` `Screen` ถือเป็นซูเปอร์คลาสของคอมโพเนนต์ส่วนติดต่อกับผู้ใช้โดยใช้ API ชั้นสูง ประกอบไปด้วย `Alert`, `Form`, `List`, `Text` ฯลฯ

`Canvas` และ `Graphic` เป็นคลาสหลักของ API ระดับต่ำ (Low-Level) แอปพลิเคชันเกมเป็นแอปพลิเคชันที่ใช้ API ระดับต่ำในการสร้างส่วนติดต่อกับผู้ใช้

2.3.8 คลาสของพื้นที่เก็บข้อมูลแบบคงตัว (Persistent Storage Class)

ในบางครั้งแอปพลิเคชันที่เขียนบน MIDP จำเป็นจะต้องเก็บข้อมูลไว้บนอุปกรณ์อย่างถาวร คลาสที่กำหนดไว้ในแพ็คเกจ `java.microedition.rms` ให้กลไกการเก็บข้อมูลถาวรที่เรียกว่า `RecordStore` โดยยอมให้แอปพลิเคชันเขียนลบและปรับปรุงบันทึกข้อมูลในหน่วยเก็บข้อมูลถาวรในอุปกรณ์

2.3.9 คลาสของเครือข่าย (Network Class)

แม้ว่าการสื่อสารที่ติดต่อสื่อสารทั่วไปที่กำหนดไว้ใน CLDC จะประกอบไปด้วยชุดของอินเทอร์เน็ตเฟสการเชื่อมต่อเครือข่าย แต่ก็ไม่มีโพรโตคอลอยู่เบื้องหลังอินเทอร์เน็ตเฟสการเชื่อมต่อจริงๆ หากแต่ปล่อยให้ให้เป็นหน้าที่ของ MIDP

ในบรรดาอินเทอร์เน็ตเฟสการเชื่อมต่อเครือข่ายเหล่านี้ `HttpConnection` ถือเป็นอินเทอร์เน็ตเฟสหลักที่ต้องมีใน MIDP เสมอ คลาสของอินเทอร์เน็ตเฟสเหล่านี้ กำหนดไว้ในแพ็คเกจ `java.microedition.io`

2.4 MIDlet

`MIDlet` (“มิดเล็ต”) หมายถึง แอปพลิเคชันบน MIDP `MIDlet` มีส่วนคล้ายกับจาวาแอปเพล็ต แม้จะไม่มีเมธอด `main()` แต่ `MIDlet` ก็นำคลาส `javax.microedition.midlet.MIDlet` ตลอดจนเมธอดทั้ง 3 [`startApp()` `pauseApp()` และ `destroyApp()`] เข้ามาใช้ นอกจากนี้ `MIDlet` ยังกำหนด constructor แบบ `public` ที่ไม่มีอาร์กิวเมนต์ใด ๆ อีกด้วย

เราสามารถให้คำนิยามของคลาส `javax.microedition.midlet.MIDlet` ได้ดังนี้

```
public abstract class MIDlet extends Object {
```

```

protected MIDlet ()
protected abstract void startApp () throws MIDletStateException
protected abstract void pauseApp ()
protected abstract void destroyApp (boolean unconditional)
    throws MIDletStateException
public final String getAppProperty (String key)
public final void notifyDestroyed ()
public final void notifyPaused ()
public final String getAppProperty (String key)
public final void resumeRequest ()
}

```

คลาส MIDlet ระบุเมธอดที่สามารถเรียกใช้โดยซอฟต์แวร์จัดการแอปพลิเคชัน (AMS) เพื่อสั่งให้แอปพลิเคชัน MIDlet เริ่มต้นและหยุดทำงาน

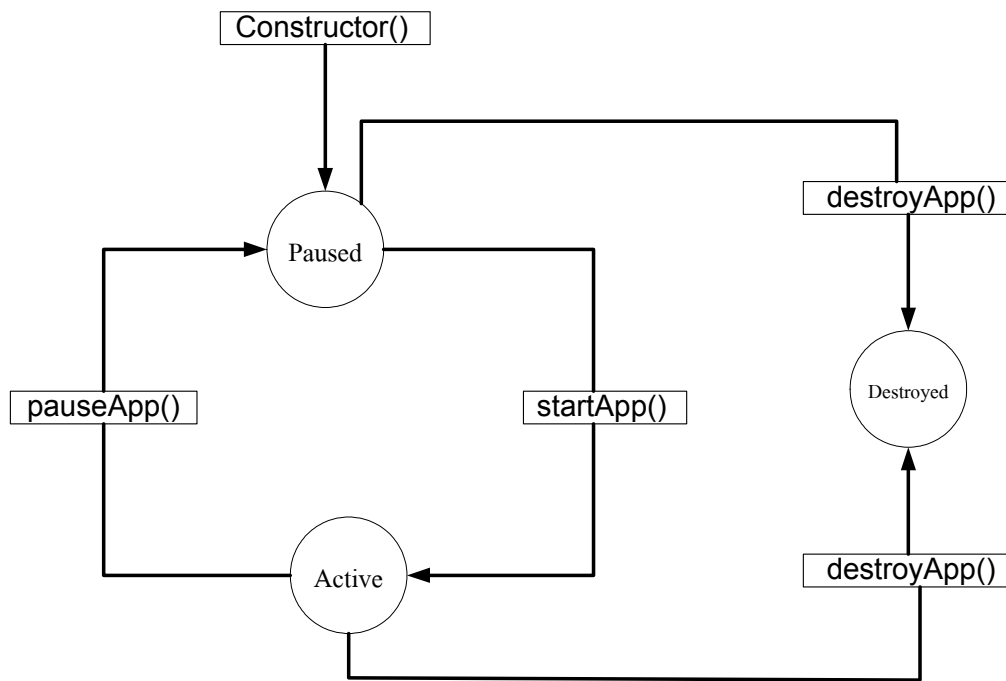
2.5 วงจรการทำงานของ MIDlet

การกระทำของ MIDlet ประกอบไปด้วย 3 สถานะ คือ กำลังทำงาน หยุดชั่วคราวและถูกทำลาย ออสซีซอฟต์แวร์จัดการแอปพลิเคชันเป็นตัวควบคุมการเปลี่ยนสถานะหนึ่งไปยังอีกสถานะหนึ่ง ควบคุมด้วยเมธอด StartApp () pauseApp() และ destroyApp()) ที่มาพร้อมกับ MIDlet รูปที่ 2-3 แสดงให้เห็นจุดเปลี่ยนระหว่างสถานะทั้ง 3 โดยการเรียกใช้เมธอดข้างต้น

เมื่อ MIDlet พร้อมสั่งการ ซอฟต์แวร์จัดการแอปพลิเคชันจะสร้างตัวอย่าง MIDlet ขึ้นมาก่อนโดยใช้ constructor แบบ public ที่ไม่มีอาร์กิวเมนต์ใดๆ โดย MIDlet จะอยู่สถานะหยุดชั่วคราว

จากนั้นซอฟต์แวร์จัดการแอปพลิเคชันจะเรียกเมธอด startApp() ขึ้นมา และ MIDlet จะเข้าสู่สถานะกำลังทำงานเปิดรับทรัพยากรที่ต้องการและเริ่มต้นการทำงาน ในสถานะนี้ MIDlet จะทำงานและดึงทรัพยากรที่ต้องการไว้ใช้งาน

เมื่อซอฟต์แวร์จัดการแอปพลิเคชันไม่ต้องการให้ MIDlet ทำงานต่อไป ก็จะเรียกเมธอด pauseApp() จากนั้น MIDlet จะหยุดทำงานและเข้าสู่สถานะหยุดชั่วคราว คืนทรัพยากรที่ดึงมาใช้งานและเข้าสู่ภาวะไม่ทำงาน MIDlet สามารถกลับไปอยู่ที่สถานะกำลังทำงาน ได้เมื่อซอฟต์แวร์จัดการแอปพลิเคชันเรียกเมธอด startApp() ขึ้นมาท้ายที่สุดเมื่อซอฟต์แวร์จัดการแอปพลิเคชันไม่ต้องการเรียกใช้งาน MIDlet หรือต้องการเคลียร์หน่วยความจำเพื่อให้โปรแกรมอื่นได้ใช้งาน ก็จะส่งสัญญาณเตรียมทำลาย MIDlet ที่ ด้วยการเรียกเมธอด destroyApp () และเข้าสู่สถานะถูกทำลาย ซึ่ง MIDlet จะปล่อยทรัพยากรทั้งหมด ทำการจัดเก็บข้อมูลถาวรต่าง ๆ และหยุดการทำงานทั้งหมด



รูปที่ 2-3 วงจรการทำงานของ MIDlet

หาก MIDlet อยู่ระหว่างสักระทำการขั้นตอนสำคัญอยู่ ก็อาจร้องขอไม่ให้เข้าสู่สถานะถูกทำลายได้โดยเรียกใช้ MIDletStateException อย่างไรก็ตาม ซอฟต์แวร์จัดการแอปพลิเคชันอาจปฏิเสธหรือยินยอมตามคำร้องขอนี้ก็ได้ ตัวแปรบูลีน unconditional ในลายเซ็นของเมธอด destroyApp () จะเป็นตัวกำกับการร้องขอนี้สมควรหรือไม่ ถ้าตัวบ่งชี้มีค่าเท่ากับ true การร้องขอก็เป็นอันตกไป ตรงกันข้ามหากมีค่าเท่ากับ false ก็จะได้รับคำตอบรับและจะเรียกเมธอด destroyApp () ขึ้นมาใหม่ในคราวต่อไป

หาก MIDlet ต้องการเข้าสู่สถานะหยุดชั่วคราวหรือถูกทำลายด้วยตัวเอง ก็สามารถทำได้โดยเรียกเมธอด notifyPause() หรือ notifyDestroy() ตามลำดับ โดยเมธอดทั้ง 2 จะแจ้งให้ซอฟต์แวร์จัดการแอปพลิเคชันทราบว่า MIDlet ได้เข้าสู่สถานะหยุดชั่วคราว/ถูกทำลายแล้ว ในกรณีนี้ซอฟต์แวร์จัดการแอปพลิเคชันจะไม่เรียกเมธอด pauseApp() หรือ destroyApp() อีก ทั้งนี้ MIDlet ควรจะล้างการใช้ทรัพยากรก่อนส่งสัญญาณให้ซอฟต์แวร์จัดการแอปพลิเคชันทราบ

MIDlet จะเข้าสู่สถานะหยุดทำงานได้ก็ต่อเมื่อผ่านสถานะกำลังทำงานมาก่อนแล้ว ในทางตรงกันข้าม หากต้องการเข้าสู่สถานะถูกทำลาย สามารถเข้าได้โดยตรงจากทั้งสถานะหยุดทำงานชั่วคราวและสถานะกำลังทำงาน นอกจากนี้ยังสามารถเข้าสู่สถานะหยุดการทำงานได้ในขณะที่กำลังทำงาน หรือเมื่อได้รับคำสั่งจากซอฟต์แวร์จัดการแอปพลิเคชัน